Databricks Data Governance — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Databricks Data Governance in Microsoft Purview 6

What this solution is . . . 6

Why this matters . . . 6

Databricks and the Purview connectors . . . 6

The building blocks . . . 7

Credentials and network — safe by design . . . 7

Licensing and the evolving portal . . . 8

What this is not . . . 8

The deployment model . . . 8

2. Requirements Guide — Databricks Data Governance (Microsoft Purview) 8

2.1 A Microsoft Purview (Data Governance) account with the Data Map / Unified Catalog available . 8

2.2 Data-plane governance roles (least-privilege) on the collection in scope . . . 9

2.3 PowerShell + Azure REST toolchain (Az + data-plane token) . . . 9

2.4 An Azure Databricks workspace and the correct connector (Unity Catalog vs. Hive metastore) . 9

2.5 A scan credential stored in Key Vault and referenced by name (no secret in code) . . . 10

2.6 An integration runtime with network line-of-sight to the workspace . . . 10

2.7 Data-governance stakeholders, classification intent, and the catalog target . . . 10

2.8 A bounded pilot scope and change control . . . 10

Checklist . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview 13

Step 1 — Assess current state (read-only) . . . 14

Step 2 — Design the collection and role model . . . 14
Step 3 — Establish the scan credential path (Key Vault, safe by design) . . . 14
Step 4 — Establish the integration runtime and network path . . . 15
Step 5 — Record the pilot scope . . . 15
Step 6 — Register the pilot Databricks source into the Data Map . . . 15
Step 7 — Configure and pilot the scan (bounded scope, pilot-first) . . . 15
Step 8 — Review scanned assets, classifications, and lineage . . . 16
Step 9 — Curate the catalog and assign ownership . . . 16
Step 10 — Validate against the pilot gate . . . 16
Step 11 — Expand across the metastore (enforce) . . . 16
5. Testing Guide — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview 17
Test 1 — Account, roles, and runtime are ready . . . 17
Test 2 — The Azure Databricks connector is available and the connector type is confirmed . . . 17
Test 3 — The scan credential resolves (Key Vault, by name) . . . 17
Test 4 — The source registers and Test connection passes . . . 18
Test 5 — The scan succeeds and returns lakehouse assets . . . 18
Test 6 — Databricks assets are classified . . . 18
Test 7 — Lineage is surfacing . . . 18
Test 8 — Scanned assets have owners and are cataloged . . . 18
Test 9 — Consolidated readiness gate . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview 19
A. Cannot reach the account / token fails . . . 19
B. The Azure Databricks connector isn't offered / wrong connector . . . 19
C. Source registration or Test connection fails . . . 20
D. Network / integration-runtime failures . . . 20
E. Scan runs but classifies nothing . . . 21
F. Lineage is missing or shallow . . . 21
G. Databricks / Azure cost higher than expected . . . 22
H. Classifications look wrong . . . 22
I. Ownership / gate not passing . . . 22
J. Diagnostics quick reference . . . 22
K. When to escalate . . . 23
7. Best Practices — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview 23

8. Deliverables Summary — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview 24
Documentation 24
Named artifacts 25
PowerShell scripts 25
What the buyer receives 26
Outcome 26
Appendix A — PowerShell Scripts 27

Executive Summary — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview

Most data-governance efforts stop at Microsoft 365, and the lakehouse — customer records, financial extracts, PII, and model features in Azure Databricks — sits outside the map, unclassified and ungoverned. You cannot protect, retain, or grant access to data you cannot see, and analysts create tables faster than any manual process can track them. This solution closes that blind spot: it **registers, scans, classifies, and catalogs your Azure Databricks / lakehouse (Unity Catalog) assets into the Microsoft Purview Data Map, with lineage**, so lakehouse data becomes a mapped, classified, owned part of the same governed estate as the rest of your data.

This is a complete, do-it-today package written for an engineer bringing Databricks into Purview for the first time. It delivers the connector workflow end-to-end — the safe **Key Vault** scan credential, the integration-runtime network path, a bounded pilot scan of one catalog/schema, classification review, **Unity Catalog lineage**, catalog curation, and ownership — with every artifact named once and reused everywhere, and every scan proven on a bounded pilot before the whole metastore is touched. Because the Purview Data Governance connectors are evolving and consumption-billed, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a connector, a portal path, or a price as permanent.

What you get

• A plain-language Solution Overview of the Databricks connectors — **Unity Catalog** (the lineage-rich

path this solution is written for) and **Hive metastore** (legacy) — how they scan and classify, and how they complement Microsoft 365 Purview.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: the Purview account,

connector confirmation, least-privilege collection roles, the Az/REST toolchain, the workspace and its metastore, the Key Vault scan credential, the integration-runtime network path, the stakeholders and classification intent, and the bounded pilot scope.

• An Implementation Guide of precise, ordered steps: design the collection/role model, establish the Key

Vault credential and network path safely, register the Databricks source, **pilot-scan** one catalog/schema, review classifications and lineage, curate the catalog and assign ownership, validate against the gate, then expand with scheduled scans.

• Five PowerShell tools for readiness/role discovery, pilot-scope and schedule capture, Databricks source

registration and scan configuration (`-WhatIf` by default), a CSV export of assets/classification/lineage coverage, and a consolidated readiness-gate check — authentic Purview data-plane REST automation with **no credentials in code** (the Databricks secret referenced by name only).

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

Databricks / lakehouse data governed from Purview: the Unity Catalog workspace registered into the Data Map, scanned and classified, cataloged with lineage, and owned — validated on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (bounded scan)** → Validate → Expand — with a pilot gate before any metastore-wide scanning.

Outcome

The lakehouse blind spot closed — Databricks catalogs, schemas, and tables discoverable, classified, lineage-wired, cataloged, and owned in the same Purview Data Map as the rest of the estate — with regulatory, records, and access determinations left to the owners who make them.

1. Solution Overview — Databricks Data Governance in Microsoft Purview

What this solution is

Your lakehouse is almost certainly your largest concentration of sensitive data — customer records, financial extracts, PII, model features — and it very often sits **outside** the governance that covers Microsoft 365. This solution governs **Azure Databricks / lakehouse** data *from* Microsoft Purview: you **register** the Databricks workspace as a data source in the Purview **Data Map**, **scan** it to enumerate and **classify** its catalogs, schemas, tables, and columns, **catalog** the assets into the Unified Catalog with owners and glossary terms, and surface **lineage** so an asset can be traced from source to consumer. The result is a lakehouse that is discoverable, classified, owned, and governed alongside the rest of your estate — not a blind spot.

> ***Boundary.*** *This solution provides the technical method to *register, scan, classify, catalog, and wire lineage* for Databricks/lakehouse assets. It does **not** make the business or regulatory determinations — which table is "confidential", what the retention/records schedule is, or who may access what. Those belong to the organization's data owners, stewards, compliance, privacy, and legal functions. The solution gives them a governed, discoverable lakehouse to decide against.*

Why this matters

Analysts and data engineers create tables in Databricks faster than any governance process can track them. A notebook joins three tables and writes a fourth; a job exports a table to a downstream report; a copy of a regulated dataset lands in a new schema. Nobody can answer "where is our PII in the lakehouse?", "who owns this table?", or "if this column changes, what breaks downstream?" — because the lakehouse was never registered, scanned, or cataloged in the tool that governs everything else. Ungoverned lakehouse data is un-securable lakehouse data. This solution registers Databricks into Purview so the lakehouse becomes part of the governed estate.

Databricks and the Purview connectors

Microsoft Purview registers and scans Azure Databricks through a **data-source connector**. There are two shapes, and which one fits depends on how your workspace is configured — **verify current availability and the exact connector for your tenant** in Microsoft's documentation, as the Data Governance experience is evolving and some connectors are newer than others:

1. **Azure Databricks Unity Catalog** — the modern connector. It reads the workspace's **Unity Catalog metastore **and enumerates its** catalogs → schemas → tables/views **into the Data Map, applies** classifications**, and brings** Unity Catalog lineage** (the table/column lineage Unity Catalog tracks) into Purview. Use this where the workspace is Unity Catalog-enabled.

1. **Azure Databricks (Hive Metastore)** — the connector for workspaces using the legacy **Hive metastore**. It enumerates and classifies Hive metastore databases/tables. Use this where Unity Catalog is not enabled.

This solution is written for the **Unity Catalog** path (the recommended, lineage-rich one) and notes where the Hive-metastore path differs. Confirm which your workspace uses before you register.

The building blocks

This solution stands up, in order:

1. **The Data Map registration** — the Databricks workspace registered as a source inside a **Purview collection** (which also carries access control). Nothing else works until the workspace is registered and reachable.

1. **Scanning & classification** — a **scoped scan** connects to the workspace, enumerates its catalogs/schemas/tables, and applies **classifications** (built-in system classifications such as credit-card or national-ID number, plus any custom classification rule you define). Classification turns the raw table inventory into a *sensitivity-aware* map.

1. **The Unified Catalog** — the business layer: a **governance domain**, one or more curated **data products **(published sets of lakehouse assets with an owner and a use purpose), a** business glossary**, and (where available)** data quality**.

1. **Lineage** — the wiring that shows how lakehouse data flows. The Unity Catalog connector brings Databricks lineage into the Data Map so you can trace an asset upstream to its origin and downstream to its consumers. Lineage availability/depth is connector- and tenant-dependent — verify.

Credentials and network — safe by design

Scanning reaches into the live Databricks workspace, so it needs a credential and a network path. This solution follows the safe-by-design pattern Microsoft documents: a **Databricks credential** (a service principal or personal access token, per the connector's requirement) is stored in an **Azure Key Vault** linked to the Purview account and referenced **by name** — **no secret is ever placed in a script or document**. Reachable (public) workspaces are scanned through the **Azure (managed) integration runtime**; **VNet-injected / private-link** workspaces are scanned through a **self-hosted integration runtime (SHIR)** with network line-of-sight. Scans are always **piloted on a bounded scope** (one catalog/schema) before the whole workspace is scanned.

Licensing and the evolving portal

Microsoft Purview Data Governance (the Data Map and Unified Catalog) is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription**, and the capability set — including specific connectors, lineage depth, and **data quality** — is evolving, with some features in preview. **Verify current pricing, connector availability, and portal paths for your tenant** in Microsoft's documentation and the Purview portal before you plan. This solution cites the method; it never asserts a price, a connector name, or a portal path as permanent.

What this is not

This is the *lakehouse data-governance* solution. It is **not** the Microsoft 365 compliance configuration (sensitivity/retention labels, DLP, insider risk, records management) — those are separate Purview solutions in this category. It is also **not** Databricks-side governance (Unity Catalog grants/ACLs remain configured in Databricks); this brings the Databricks estate *into* Purview so it is mapped, classified, cataloged, and lineage-wired next to everything else.

The deployment model

> *Assess the workspace and readiness, design the collection/domain/role model, configure the source and scan safely, **pilot the scan on one catalog/schema**, validate against the gate, then expand across the metastore — enabling scheduled scans only after a scope passes. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot (bounded scan)** → **Validate** → **Expand.***

2. Requirements Guide — Databricks Data Governance (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance/governance licensing, least-privilege Purview roles, the PowerShell + Graph/REST toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

> ***Read this first.** Governing Databricks is a **Microsoft Purview Data Map / Unified Catalog** capability (the data-governance side), **not** a Microsoft 365 E5 compliance feature. Its prerequisites are a **Purview (Data Governance) account**, **data-plane roles**, a **Databricks workspace with a reachable credential**, and an **integration runtime with network line-of-sight** — different from the compliance-side prerequisites. Everything below is **cited as of authoring and flagged "verify in your tenant"**; the Databricks connector(s), provisioning, pricing, roles, portal paths, and REST* `api-version` *values in this area are actively evolving. Nothing is invented.*

2.1 A Microsoft Purview (Data Governance) account with the Data Map / Unified Catalog available

Description: A provisioned **Microsoft Purview account** (Data Map / Unified Catalog) under an Azure subscription, reachable at its data-plane endpoint `https://<account>.purview.azure.com`. **Why:**

The Databricks workspace is registered, scanned, and cataloged **inside** this account; without it there is nothing to register the lakehouse into (extends Global Requirement 1 to the governance side). **Verify:** In the **Purview portal** confirm the account/catalog loads and the **Data Map** is present; `DGDBX-Set-Scope.ps1` confirms the account endpoint is reachable and a data-plane token is obtainable. **Implement:** Provision a Purview (Data Governance) account (Azure portal / `Az.Purview`) under change control. **Verify the current provisioning model and pricing** — Data Governance is Azure-metered / consumption-based (pay-as-you-go) — against Microsoft's documentation before creating it.

2.2 Data-plane governance roles (least-privilege) on the collection in scope

Description: The **least-privilege Purview data-plane roles**, assigned **at the collection scope** where Databricks will be registered: **Collection admin** (create/manage the collection), **Data source admin** (register the source, manage scan credentials, run scans), and **Data curator** (read the catalog, edit assets/classifications, read lineage). Reserve root-collection admin for the few setup actions (Global Requirement 2). **Why:** Purview Data Map uses **collection-scoped** data-plane roles distinct from the M365 compliance role groups; registering the workspace, running the scan, and editing catalog assets each require the matching role at the right collection. **Verify:** Purview portal → **Data Map** → **Collections** → `Lakehouse – DGDBX – Pilot` → **Role assignments** (verify path) shows your account in the required roles; `DGDBX-Set-Scope.ps1` reports the collections your token can enumerate. **Implement:** Assign the least-privilege data-plane roles at the pilot collection first, root-collection admin only where genuinely required, under change control.

2.3 PowerShell + Azure REST toolchain (Az + data-plane token)

Description: A workstation with the **Az PowerShell** modules (`Az.Accounts`, and `Az.Purview` for control-plane account operations), a modern-auth/MFA admin, and the ability to acquire a **Purview data-plane token** via `Get-AzAccessToken -ResourceUrl 'https://purview.azure.net'` for `Invoke-RestMethod` calls to the scanning/catalog APIs (Global Requirement 3). **Why:** The provided scripts read and (with `-Commit`) register/scan Databricks through the documented Purview data-plane REST APIs, authenticated with an Entra ID token — no credentials in code. **Verify:** `Get-Module -ListAvailable Az.Accounts, Az.Purview`; `Connect-AzAccount` succeeds and `Get-AzAccessToken -ResourceUrl 'https://purview.azure.net'` returns a token. `DGDBX-Set-Scope.ps1` confirms connectivity. **Implement:** `Install-Module Az.Accounts, Az.Purview -Scope CurrentUser`; sign in with a modern-auth admin; **verify the current data-plane resource URL /** `api-version` **values** in Microsoft's docs.

2.4 An Azure Databricks workspace and the correct connector (Unity Catalog vs. Hive metastore)

Description: The **Azure Databricks workspace** to be governed, and confirmation of **which connector applies** — **Azure Databricks Unity Catalog** (workspace is Unity Catalog-enabled; the lineage-rich path this solution is written for) or **Azure Databricks (Hive metastore)** (legacy). You must know the workspace URL and, for Unity Catalog, the **metastore** it is attached to. **Why:** The registration shape, what gets enumerated (Unity Catalog `catalogs` → `schemas` → `tables` vs. Hive databases/tables), and whether lineage flows depend on the connector. Choosing the wrong one produces an empty or partial scan. **Verify:** In the Databricks admin console confirm whether Unity Catalog is enabled and the metastore;

verify the current connector name and availability for your tenant in Microsoft's documentation (the Data Governance connector set is evolving). **Implement:** Confirm the workspace's catalog architecture with the Databricks admins before registering; select the matching Purview connector.

2.5 A scan credential stored in Key Vault and referenced by name (no secret in code)

Description: The **credential** the scan uses to authenticate to Databricks — a **service principal** or **personal access token (PAT)** per the connector's requirement — stored as a secret in an **Azure Key Vault linked to the Purview account** and referenced in Purview **by name**. The credential's Databricks-side identity needs read/metadata access to the catalogs/schemas in scope. **Why:** Scanning reaches into the live workspace, so it needs a credential and least-privilege access; Microsoft's documented pattern keeps the secret in Key Vault and never in a script or document — **no secret is ever placed in code here**. **Verify:** The Key Vault connection exists in Purview (**Management** → **Credentials / Key Vault connections**, verify path) and the credential resolves; the Databricks identity can read the in-scope catalogs. `DGDBX-Set-Scope.ps1` confirms a named Key Vault connection is present (it never reads the secret value). **Implement:** Create the Key Vault connection and store the Databricks secret; grant the scan identity least-privilege metadata read in Databricks (Unity Catalog grants / workspace ACLs) before scanning.

2.6 An integration runtime with network line-of-sight to the workspace

Description: The **integration runtime (IR)** the scan runs on: the **Azure (managed) IR** for a **reachable/public** workspace, or a **self-hosted integration runtime (SHIR)** on a host with network line-of-sight for a **VNet-injected / Private Link** workspace. **Why:** The scan connects from the IR to the workspace; a private workspace is unreachable from the managed IR, so the scan fails or times out without a SHIR that can route to it. **Verify:** Confirm the workspace's network posture (public vs. private) with the Databricks/network admins; if private, a SHIR (`SHIR – DGDBX`) is installed and shows **running**. `DGDBX-Set-Scope.ps1` reports whether a self-hosted IR is registered when one is required. **Implement:** Install and register a SHIR on an in-network host for a private workspace; otherwise the Azure managed IR is used. **Verify current IR requirements for the connector** in Microsoft's docs.

2.7 Data-governance stakeholders, classification intent, and the catalog target

Description: Engagement with the **data-governance, records, privacy, and business-data owners** (Global Requirement 5) who define **which catalogs/schemas are in scope**, the **classification intent** (which built-in/custom classifications matter for this lakehouse), and the **catalog target** (which assets become curated **data products** with owners and glossary terms). **Why:** Purview provides the technical map; **which table is "confidential", what the retention/records obligation is, and who may access what are business/regulatory determinations** the owners make — not technical defaults. **Verify:** Named owners, an in-scope catalog/schema list, and a classification/catalog intent are recorded. **Implement:** Identify the owners and capture scope + intent before scanning; you deliver the governed, classified, cataloged lakehouse and the evidence — they own the determinations.

2.8 A bounded pilot scope and change control

Description: A **bounded pilot scan scope** (`Lakehouse Scan – DGDBX – Stage1` limited to **one catalog/schema**) and a **change-control** process, per Global Requirement 6. **Why:** A first full-workspace scan against a large lakehouse is high-impact (load, cost, and blast radius); it is proven on one catalog/schema first. The model is **Assess** → **Design** → **Configure** → **Pilot (bounded scan)** → **Validate** → **Expand**. **Verify:** The pilot scope (one catalog/schema) is named and a change window/approval path exists. **Implement:** Define the bounded pilot scope and agree change control before scanning the whole metastore or enabling scheduled scans.

Checklist

- Microsoft Purview (Data Governance) account provisioned; Data Map / Unified Catalog available (pricing/model verified)
- Data-plane roles (Collection admin / Data source admin / Data curator) assigned least-privilege at `Lakehouse – DGDBX – Pilot`
- `Az.Accounts` + `Az.Purview` installed; `Connect-AzAccount` and `Get-AzAccessToken` (purview.azure.net) succeed
- Databricks workspace identified; connector confirmed (Unity Catalog vs. Hive metastore)
- Scan credential (service principal / PAT) stored in **Key Vault**, referenced by name; Databricks-side read access granted
- Integration runtime with line-of-sight (Azure managed for public; `SHIR – DGDBX` for private)
- Governance/business owners named; in-scope catalogs/schemas, classification intent, and catalog target agreed
- `Lakehouse Scan – DGDBX – Stage1` bounded to one catalog/schema; change control agreed

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview

Follow these steps in order. This guide assesses Databricks readiness, designs the collection and role model, establishes the scan credential and network path safely, registers and **pilot-scans** a bounded Databricks scope (one catalog/schema), reviews classifications and lineage, then expands across the metastore. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Every scan is piloted on a bounded scope before the whole workspace is scanned.**

> *Model: prove the Databricks connector, the credential/network path, classification, and lineage on a pilot scope, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot (bounded scan)** → **Validate** → **Expand.** Verify current portal paths, the exact connector name (Azure Databricks*

Unity Catalog vs. Hive metastore), connector prerequisites, lineage support, and the data-plane `api-version` *values for your tenant as you go — the Data Governance experience is evolving.*

Step 1 — Assess current state (read-only)

1. Run `DGDBX-Discovery.ps1` to capture readiness signals: the Purview account endpoint is reachable,

which **collections** exist and their **role coverage** (especially Data source admin, needed to register/scan), which **integration runtimes** are registered, and how many **Azure Databricks / lakehouse** sources are already registered. No changes are made.

1. In the Purview portal (**Data Map** → **Sources** → **Register**) confirm the **Azure Databricks** source

type is offered and note **which connector applies** — **Unity Catalog** (this solution's path) or **Hive metastore** — for your tenant. **Verify the exact connector name/availability**; it is evolving.

1. With the Databricks admins, confirm whether the workspace is **Unity Catalog-enabled** and the

metastore it attaches to, the **workspace URL**, and the **network posture** (public vs. VNet-injected / Private Link).

1. Record gaps against the Requirements checklist. Do **not** proceed past a missing account, a missing

connector, missing least-privilege roles, a missing scan credential, or a missing network path — remediate those first.

Step 2 — Design the collection and role model

1. Decide the **collection** the Databricks source registers into. For the pilot, create one collection

named `DGDBX-Collection-Pilot` under the root; plan the broader hierarchy (for example a `Lakehouse` branch by workspace, business unit, or sensitivity) on paper for the expansion phase.

1. Assign least-privilege data-plane roles at the narrowest scope: **Data source admin** on

`DGDBX-Collection-Pilot` to whoever will register/scan the workspace, **Data curator** to the catalog editors (edit assets/classifications, read lineage), **Data reader** to browsers, and **Collection admin** to a small named group. Reserve root-collection admin for the few setup actions only.

Step 3 — Establish the scan credential path (Key Vault, safe by design)

1. Link an **Azure Key Vault** to the Purview account (or confirm one is linked).

2. Provision the **Databricks scan identity** the connector requires — a **service principal** or a

personal access token (PAT), per the connector's requirement (**verify which your connector accepts**). Grant it **least-privilege metadata read** on the in-scope catalogs/schemas in Databricks (Unity Catalog grants such as `USE CATALOG` / `USE SCHEMA` / `SELECT` or the metadata equivalents the connector documents — verify), never workspace-admin.

1. Store the secret in the **Key Vault** and create a Purview credential **by name** — `DGDBX-Cred-Databricks-KV`

— that references the Key Vault secret. **No credential is ever placed in a script or document.** This name is what the scripts reference; the secret is never inlined.

Step 4 — Establish the integration runtime and network path

1. Confirm the **integration runtime** for the workspace: the **Azure (managed) integration runtime** for

a **reachable/public** workspace, or install and register a **self-hosted integration runtime (SHIR)** named `DGDBX-SHIR` on an in-network host for a **VNet-injected / Private Link** workspace.

1. Confirm the runtime shows **Running/Available**, and open only the network rules the scanner needs to

reach the workspace (and the Unity Catalog metastore endpoint) — no broader. **Verify current IR / networking requirements for the connector** in Microsoft's docs.

Step 5 — Record the pilot scope

1. Run `DGDBX-Set-Scope.ps1` to record the pilot Databricks scope — the **workspace URL**, the **one

catalog/schema **to scan first, the owner/steward, the collection, the credential name, the scan rule set intent, and the (previewed)** schedule**. It writes a local scope manifest and, by default, only** previews** the schedule; it changes nothing in the tenant unless you pass `-Commit`.

Step 6 — Register the pilot Databricks source into the Data Map

1. Register the workspace into `DGDBX-Collection-Pilot`. You may use the portal

(**Register** → **Azure Databricks**), or run `DGDBX-Register-DatabricksSource.ps1` which **defaults to** `-WhatIf` and prints exactly what it would register (the connector kind, the workspace URL, the metastore, the collection, and the credential name); re-run with `-Commit` to register.

1. Run **Test connection** on the registered source and confirm it passes (the scan identity authenticates

to the workspace, the metastore is reachable over the chosen IR) before scanning. Do not scan until Test connection passes.

Step 7 — Configure and pilot the scan (bounded scope, pilot-first)

1. Define the scan configuration: a **scan rule set** named `DGDBX-ScanRuleSet-Pilot` (start from the

system default rule set), the **classifications** to apply (system classifications plus any custom rule named `DGDBX-Classification-Custom-<name>`), and a **scoped** scan that targets only the pilot **catalog/schema** — not every catalog in the metastore.

1. Trigger the scan **once** on the bounded scope (not scheduled yet) via

`DGDBX-Register-DatabricksSource.ps1 -ConfigureScan` (`-WhatIf` by default; `-Commit` to apply) or the portal. Pass/fail: the scan completes with **status Succeeded** and returns assets (catalogs → schemas → tables/views) with classifications.

1. **Rollback:** if a scan misbehaves, over-reaches, or runs up Databricks/Azure cost, cancel the run in

the portal and set the scan to disabled; a scan does not modify source data, but cancel to stop resource/cost consumption. Re-scope and re-run on a smaller set (a single schema).

Step 8 — Review scanned assets, classifications, and lineage

1. Run `DGDBX-Report.ps1` to export the pilot Databricks assets discovered (catalogs, schemas,

tables/columns), the classifications applied, any assets returned **unclassified**, and **lineage coverage** (assets with lineage relationships) to CSV.

1. With the data owner, confirm the classifications are correct. Where a real internal pattern was missed,

define a **custom classification rule** (`DGDBX-Classification-Custom-<name>`), validate it against sample data, add it to `DGDBX-ScanRuleSet-Pilot`, and re-scan the pilot scope.

1. Confirm **Unity Catalog lineage** is surfacing in the Data Map (table/column lineage where Unity

Catalog tracks it). **Lineage availability and depth are connector- and tenant-dependent — verify**; if lineage is absent, confirm the connector version and Unity Catalog lineage prerequisites before assuming a defect.

Step 9 — Curate the catalog and assign ownership

1. Assign the **data owner** and **steward** recorded in the scope manifest to the scanned catalog/schema/

tables so every governed lakehouse asset has a named owner — ownership is what makes classification and later access decisions accountable.

1. In the **Unified Catalog**, create the governance domain `DGDBX-Domain-Lakehouse` and publish the

pilot assets as a curated **data product** (`DGDBX-DataProduct-<name>`) with an owner, a use purpose, and glossary terms. **Verify current Unified Catalog paths and data-quality availability for your tenant** — the catalog experience is evolving and some capabilities are in preview.

Step 10 — Validate against the pilot gate

1. Complete the Testing Guide. Run `DGDBX-Test-Readiness.ps1` for a consolidated PASS/GAP gate check. The

pilot passes only when: the account is reachable and roles are assigned; the integration runtime is present and the Key Vault credential resolves; the Databricks source is registered and **Test connection** passes; the scan **Succeeded** and returned classified assets; and every scanned pilot asset has an owner assigned.

1. Remediate any gaps and re-run the relevant validation before proceeding. Do **not** widen scope until

the gate passes.

Step 11 — Expand across the metastore (enforce)

1. After the pilot passes, register/scan further catalogs and schemas one grouping at a time, re-running

`DGDBX-Set-Scope.ps1`, `DGDBX-Register-DatabricksSource.ps1`, and `DGDBX-Report.ps1` for each new scope. Reuse the same Key Vault credential pattern (extending the Databricks-side metadata read grants to the new catalogs/schemas).

1. Move scans from one-time to **scheduled** only once a scope is validated — set the schedule with

`DGDBX-Set-Scope.ps1 -Commit` (or in the portal) — so classifications and lineage stay current as lakehouse data changes. Watch Databricks compute / Azure metered cost as scope grows.

1. Keep ownership assigned on every newly scanned asset, keep curating data products, and track coverage

(classification and lineage) with `DGDBX-Report.ps1`.

Order of operations

```
Assess -> Design (collection/roles) -> Scan credential (Key Vault) ->
Integration runtime & network -> Record scope -> Register Databricks source ->
Pilot scan (bounded catalog/schema) -> Review classifications + lineage ->
Curate catalog & assign ownership -> Validate gate -> Expand (scheduled scans)
```

5. Testing Guide — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview

Run these tests after the pilot scan (Implementation Steps 6–9) and again after each expansion scope. Each test confirms one part of the Databricks-governance workflow works end to end. All read-only checks are safe to repeat.

Test 1 — Account, roles, and runtime are ready

Do: Run `DGDBX-Discovery.ps1`. Expected: The Purview account endpoint is reachable, `DGDBX-Collection-Pilot` exists, least-privilege roles (Collection admin, Data source admin, Data curator, Data reader) are reported as assigned — **Data source admin** (needed to register/scan) is present — and at least one **integration runtime** is registered. No required role is missing.

Test 2 — The Azure Databricks connector is available and the connector type is confirmed

Do: In the Purview portal, open **Data Map** → **Sources** → **Register**. Expected: **Azure Databricks** appears as a registerable source type for your tenant, and the connector that matches your workspace — **Unity Catalog** (this solution's path) or **Hive metastore** — is the one you register. Confirm the workspace's Unity Catalog / metastore status with the Databricks admins.

Test 3 — The scan credential resolves (Key Vault, by name)

Do: In Purview **Management** → **Credentials / Key Vault connections**, confirm `DGDBX-Cred-Databricks-KV` references the Key Vault secret **by name**; `DGDBX-Discovery.ps1` reports the Key Vault connection present. Expected: The credential resolves and the Databricks-side scan

identity (service principal / PAT) can read the in-scope catalogs' metadata. **No secret is stored in any script or document.**

Test 4 — The source registers and Test connection passes

Do: On the registered Databricks source, run **Test connection**; confirm the integration runtime shows Running/Available and the credential references the Key Vault secret by name. Expected: Purview authenticates to the workspace using the **named Key Vault credential** over a working network path (Azure managed IR, or `DGDBX-SHIR` for a private workspace) and **Test connection passes**.

Test 5 — The scan succeeds and returns lakehouse assets

Do: Review the pilot scan run in the portal and run `DGDBX-Report.ps1`. Expected: The scan status is **Succeeded**, the run scanned the intended bounded scope (the pilot catalog/schema, not the whole metastore), and assets (catalogs → schemas → tables/views/columns) were returned into the Data Map.

Test 6 — Databricks assets are classified

Do: Run `DGDBX-Report.ps1` and review the classification coverage for the pilot assets. Expected: System (and any custom) classifications are applied to the assets that contain matching data; the count of unexpectedly **unclassified** assets is understood and accepted by the data owner.

Test 7 — Lineage is surfacing

Do: Open a scanned table in the catalog and review its lineage; run `DGDBX-Report.ps1` for lineage coverage. Expected: Where Unity Catalog tracks it, **table/column lineage** is visible in the Data Map (upstream to origin, downstream to consumers). **Lineage depth is connector- and tenant-dependent — verify**; absent lineage is investigated (connector version / Unity Catalog lineage prerequisites), not assumed a defect.

Test 8 — Scanned assets have owners and are cataloged

Do: Open the pilot catalog/schema in the catalog and run `DGDBX-Report.ps1`. Expected: Every scanned pilot asset has a named **owner/steward** assigned (from the scope manifest); the pilot assets are published under the `DGDBX-Domain-Lakehouse` governance domain as a data product. No governed lakehouse asset is unowned.

Test 9 — Consolidated readiness gate

Do: Run `DGDBX-Test-Readiness.ps1 -PurviewAccount <account> -Collection DGDBX-Collection-Pilot`. Expected: A single PASS/GAP summary confirming account reachable, roles present, integration runtime registered, Databricks source registered and Test-connection-ready, at least one scan Succeeded, classification coverage > 0, and owners assigned — the pilot gate in one view.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–8 pass: connector available and confirmed, credential resolves, Test connection passes, scan Succeeded with classified assets, lineage reviewed, and every scanned asset has an owner (Test 9 shows all PASS)
Expand pass	Tests 1–8 pass for each new catalog/schema scope, and scheduled scans are enabled only after the scope validates (with expected Databricks/Azure cost accepted)

6. Troubleshooting Guide — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview

Use `DGDBX-Discovery.ps1` for readiness/role/runtime/Key-Vault and Databricks-source state, `DGDBX-Report.ps1` for scan status and classification/ownership/lineage coverage (CSV), `DGDBX-Set-Scope.ps1` to (re)record a scope and (with `-Commit`) set a schedule, `DGDBX-Register-DatabricksSource.ps1` (`-WhatIf` by default) to register/scan, and `DGDBX-Test-Readiness.ps1` for the consolidated readiness/pilot gate. All scripts are read-only unless you pass `-Commit` (only `DGDBX-Register-DatabricksSource.ps1` and `DGDBX-Set-Scope.ps1 -Commit` write).

A. Cannot reach the account / token fails

Symptom	Cause	Remediation
`Discovery` reports endpoint GAP	Wrong account name or endpoint	Confirm the account name; the endpoint is `https://<account>.purview.azure.com`; re-run `DGDBX-Discovery.ps1`
Token acquisition fails	Not signed in / wrong resource	Run `Connect-AzAccount`; the scripts request a token for `https://purview.azure.net` — confirm your account can obtain it
401/403 on every call	Missing least-privilege role	Assign Data reader (browse) or the specific role the action needs at the collection scope; re-run
Calls fail with "api-version" errors	The example API version has moved	The Data Governance experience is evolving — verify and set the current data-plane API version for your tenant

B. The Azure Databricks connector isn't offered / wrong connector

Symptom	Cause	Remediation
No **Azure Databricks** in Register	Connector unavailable/preview for your tenant	Verify licensing/region and Microsoft's current Data Governance connector availability; scope the pilot to what is actually offered
Empty or partial scan	Wrong connector for the workspace (Unity Catalog vs. Hive metastore)	Confirm with the Databricks admins whether Unity Catalog is enabled and the metastore; register the **matching** connector
Unknown source "kind"	Connector name differs from the script default	Verify the exact `kind`/`credentialType` names for your tenant before `-Commit` (the `-Kind` param accepts Unity Catalog or Hive metastore)

C. Source registration or Test connection fails

Symptom	Cause	Remediation
Register returns 403	Not a Data source admin on the collection	Assign Data source admin on `DGDBX-Collection-Pilot`; re-run `DGDBX-Register-DatabricksSource.ps1 -Commit`
Test connection fails: auth	Wrong/expired Databricks service principal or PAT, or Key Vault secret	Confirm the scan identity and the Key Vault secret; reference the credential **by name** (`-Credential DGDBX-Cred-Databricks-KV`); re-test
Test connection fails: access denied on catalog/schema	Scan identity lacks Databricks metadata read	Grant least-privilege Unity Catalog metadata read (e.g. `USE CATALOG`/`USE SCHEMA`/`SELECT` or the connector's documented equivalents) to the scan identity; re-test
"Credential not found" / Key Vault error	Vault not linked or secret name wrong	Link the Key Vault to the account, store the secret, reference it **by name**; never inline a token or secret
Metastore not resolved (Unity Catalog)	Wrong/missing `-Metastore`	Confirm the workspace's Unity Catalog metastore with the Databricks admins; verify the required metastore value for the connector; re-register

D. Network / integration-runtime failures

Symptom	Cause	Remediation
Test connection times out	Workspace is VNet-injected / Private Link, no path from managed IR	Install and register a self-hosted integration runtime (`DGDBX-SHIR`) with line of sight; open only the rules the scanner needs to reach the workspace + metastore; re-test
`Discovery` shows no integration runtime	No IR registered	Register the Azure (managed) IR for a public workspace, or a SHIR for a private one; re-run `DGDBX-Discovery.ps1`
Intermittent scan failures on a private workspace	SHIR host offline / network rule changed	Confirm the SHIR shows Running/Available and the network path is open; re-run the scan

E. Scan runs but classifies nothing

Symptom	Cause	Remediation
Assets returned, 0 classified	Scan rule set has no classifications	Confirm `DGDBX-ScanRuleSet-Pilot` includes system classifications; re-scan and review with `DGDBX-Report.ps1`
An internal pattern isn't detected	No custom classification rule	Define `DGDBX-Classification-Custom-<name>`, validate against sample data, add to the rule set, re-scan
Scan scope too broad / slow / costly	Whole metastore scanned	Re-scope the scan to one catalog/schema (`-Catalog`/`-Schema`); keep it one-time until validated
Scan status Failed	Transient/auth/network	Open the run in the portal for the failure reason; fix credential/runtime/network; re-run

F. Lineage is missing or shallow

Symptom	Cause	Remediation
No lineage on scanned tables	Connector/tenant doesn't surface Unity Catalog lineage yet, or Unity Catalog lineage prerequisites unmet	**Verify** the connector version and Unity Catalog lineage prerequisites for your tenant before assuming a defect; confirm lineage is tracked in Unity Catalog for those tables
`DGDBX-Report.ps1` shows 0% lineage	Lineage API version/shape differs, or lineage not yet computed	Verify the current lineage API version for your tenant; allow Unity Catalog lineage to populate; re-run the report

Symptom	Cause	Remediation
Partial lineage	Depth is connector-dependent	Lineage depth varies by connector and evolves — do not promise coverage the connector doesn't provide; verify current support

G. Databricks / Azure cost higher than expected

- Scanning drives Databricks compute and Purview Data Governance is consumption-billed. Keep scans

one-time and bounded until validated, scope to one catalog/schema, and agree the expected cost with the workspace and Azure subscription owners as part of change control. Cancel a runaway scan in the portal (no source data is modified) and re-scope. Enable a schedule (`DGDBX-Set-Scope.ps1 -Commit`) only after a scope validates.

H. Classifications look wrong

- False positives/negatives are expected on first scan. Review with the **data owner** using

`DGDBX-Report.ps1`; tune or remove a custom classification rule and re-scan the pilot scope. The solution provides the technical result — the owner decides whether a classification is correct for the business.

I. Ownership / gate not passing

Symptom	Cause	Remediation
`DGDBX-Test-Readiness.ps1` shows "not owned"	Owners not assigned on scanned assets	Assign the owner/steward from the scope manifest to each catalog/schema/table in the catalog; re-run
Gate shows no Succeeded scan	Scan not run or Failed	Trigger the one-time scan; fix any Test-connection failure first; re-check
Gate shows classification 0	Rule set / scope issue	See section E; re-scan and re-check
Gate shows Key Vault / IR GAP	Vault not linked / no runtime	Link the Key Vault and register the integration runtime; re-run `DGDBX-Test-Readiness.ps1`

J. Diagnostics quick reference

```
.\DGDBX-Discovery.ps1 -PurviewAccount contoso-purview                # readiness: endpoint, collections,
.\DGDBX-Set-Scope.ps1 -Collection DGDBX-Collection-Pilot -WorkspaceUrl https://adb-1234567890123456.7.azur
    -Catalog contoso_prod -Schema finance -Credential DGDBX-Cred-Databricks-KV   # record scope (preview;
.\DGDBX-Register-DatabricksSource.ps1 -PurviewAccount contoso-purview -Collection DGDBX-Collection-Pilot `
    -SourceName dbx-lakehouse-01 -WorkspaceUrl https://adb-1234567890123456.7.azuredatabricks.net `
    -Metastore contoso-uc-metastore -Credential DGDBX-Cred-Databricks-KV   # -WhatIf by default
```

```
.\DGDBX-Report.ps1 -PurviewAccount contoso-purview -Collection DGDBX-Collection-Pilot -OutFile .\dgdbx-rep
.\DGDBX-Test-Readiness.ps1 -PurviewAccount contoso-purview -Collection DGDBX-Collection-Pilot  # consolida
```

K. When to escalate

If the account is reachable, roles are assigned, the runtime is Running, the Key Vault credential resolves, Test connection passes, and a scoped scan still returns no classified assets after tuning the rule set, capture the `DGDBX-Discovery.ps1` and `DGDBX-Report.ps1` output and the portal scan-run detail, and review the source connectivity and rule set with the data owner and your Purview/Azure administrator (and your Databricks workspace admin for Unity Catalog grants / metastore / network). For evolving or preview connectors and lineage, verify current availability, connector name, and API versions against Microsoft's documentation before assuming a defect.

7. Best Practices — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview

- Confirm the connector before you design anything. The Azure Databricks connector — **Unity Catalog** vs.

Hive metastore — its prerequisites, and its lineage support are evolving; read them from the Purview registration UI and Microsoft's current docs for your tenant rather than assuming a connector name or a capability that may have moved. Choosing the wrong connector produces an empty or partial scan.

- Prefer Unity Catalog where it is enabled. The Unity Catalog connector enumerates `catalogs → schemas →

tables/views` and brings Unity Catalog **lineage** into Purview — the lineage-rich path. Use the Hive metastore connector only for workspaces that are not Unity Catalog-enabled.

- Keep the scan secret in Key Vault, referenced by name. Link an Azure Key Vault to the account, store the

Databricks service principal / PAT there, and reference it by name (`DGDBX-Cred-Databricks-KV`) — this is both the secure pattern and what keeps the provided scripts secret-free. **No Databricks token or secret ever goes into a script, a scan, or a document.**

- Grant the scan identity least-privilege metadata read — never workspace admin. The connector needs to

read catalog/schema/table **metadata** for the in-scope objects (Unity Catalog grants / metadata read), not admin rights and not row data beyond what classification sampling requires. Widen the grants deliberately as you expand to new catalogs.

- Pilot every scan on a bounded scope. Scoped, one-time scans on a single catalog/schema prove

classification, connectivity, lineage, and **Databricks/Azure cost** before you point a scan at an entire metastore; move to scheduled scans only after a scope validates.

- Mind compute and metered cost. Scanning drives Databricks compute and Purview Data Governance is

consumption-billed; scope scans tightly, keep them one-time until validated, and agree the expected cost with the workspace and Azure subscription owners as part of change control. Cancel a runaway scan in

the portal (no source data is modified) and re-scope.

• Trust system classifications first, add custom sparingly. The built-in classifications cover most

regulated data; define a custom classification rule (`DGDBX-Classification-Custom-<name>`) only for a real internal pattern, and validate it against sample data before metastore-wide use.

• Treat lineage as verify-first, not guaranteed. Unity Catalog lineage depth in Purview is connector- and

tenant-dependent and evolving; confirm what surfaces for your tenant and connector version, and don't promise lineage coverage the connector doesn't yet provide.

• Every scanned lakehouse asset gets an owner and a steward, and is cataloged. Ownership is what makes

classification and access decisions accountable; publish curated **data products** under `DGDBX-Domain-Lakehouse` so the lakehouse is discoverable with a use purpose — an unowned, uncataloged table is a governance gap, not a governed asset.

• Right-size the network path. Use the Azure (managed) integration runtime for a reachable workspace; stand

up a self-hosted integration runtime (`DGDBX-SHIR`) with line of sight for a VNet-injected / Private Link workspace, and open only the rules the scanner needs to reach the workspace and metastore — no broader.

• Treat determinations as someone else's job. The solution registers, scans, classifies, catalogs, and

wires lineage for Databricks data; the business/records/privacy/legal owners decide sensitivity, retention, and access. Keep that separation explicit so technical scanning is never mistaken for a compliance ruling.

• Verify licensing, pricing, and portal paths as you go. The Data Governance connectors, the Unified

Catalog experience, and consumption billing are evolving — confirm current availability, connector name, lineage support, `api-version` values, and pricing in Microsoft's docs and the portal rather than assuming a path or a price is permanent.

8. Deliverables Summary — Databricks Data Governance (Unity Catalog / lakehouse): Microsoft Purview

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Registering, scanning, classifying, and cataloging Azure Databricks / lakehouse (Unity Catalog) assets into the Purview Data Map with lineage — the connectors, the safe credential/network model, and how it complements M365 Purview (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered steps: design, Key Vault credential + network, register, pilot-scan, classify, review lineage, curate catalog, assign ownership, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Pilot collection	`DGDBX-Collection-Pilot`
Databricks scan credential (references the Key Vault secret by name)	`DGDBX-Cred-Databricks-KV`
Scan rule set	`DGDBX-ScanRuleSet-Pilot`
Custom classification rule (as needed)	`DGDBX-Classification-Custom-<name>`
Self-hosted integration runtime (private workspace)	`DGDBX-SHIR`
Governance domain (Unified Catalog)	`DGDBX-Domain-Lakehouse`
Curated data product (as published)	`DGDBX-DataProduct-<name>`

PowerShell scripts

Script	What it does
`DGDBX-Discovery.ps1`	Reports readiness: account endpoint reachable, collections + role coverage, registered integration runtimes, Key Vault connection presence, and registered Azure Databricks / lakehouse source counts (Purview data-plane REST via an Azure token; read-only)
`DGDBX-Set-Scope.ps1`	Records the pilot/expansion Databricks scope — workspace, in-scope catalog/schema, owners, collection, credential name, scan-rule-set intent, and schedule — as a scope manifest; previews the schedule by default, applies it only with `-Commit` (read-only unless `-Commit`)
`DGDBX-Register-DatabricksSource.ps1`	Registers an Azure Databricks (Unity Catalog / lakehouse) source and configures a scoped scan; **defaults to** `-WhatIf` (prints the plan), applies only with `-Commit`; credential referenced by name — no secrets in code

Script	What it does
`DGDBX-Report.ps1`	Exports cataloged Databricks assets, classification results, and lineage coverage to CSV: per-source scan status, classified vs unclassified assets, and lineage/ownership coverage (read-only)
`DGDBX-Test-Readiness.ps1`	Consolidated PASS/GAP gate across account reachability, roles, integration runtime, credential, Databricks source registration/Test-connection readiness, scan status, classification coverage, and ownership (read-only)

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Databricks / lakehouse data governed from Purview — the Unity Catalog workspace registered into the Data Map, scanned, classified, cataloged with lineage, and owned — proven on a bounded pilot before you scale, with the lakehouse blind spot closed and regulatory, records, and access determinations left to the owners who make them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGDBX-Discovery.ps1

Purpose: Reports readiness so you can assess current state before registering the Databricks workspace or scanning: whether the Microsoft Purview account data-plane endpoint is reachable, which collections exist and their role-assignment coverage (especially Data source admin, needed to register/scan), which integration runtimes are registered, whether a Key Vault connection is present, and how many Azure Databricks / lakehouse sources are already registered. Read-only — it makes no change to the account, sources, or catalog. Supports the Assess step. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGDBX-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes, Key Vault connection presence, and a count of registered Azure Databricks / lakehouse sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Databricks Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the Azure Databricks connector (Unity Catalog vs. Hive metastore) are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGDBX-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes, Key Vault connection presence, and a count of registered Azure Databricks / lakehouse sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Databricks Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the Azure Databricks connector (Unity Catalog vs. Hive metastore) are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGDBX-Register-DatabricksSource.ps1

Purpose: Registers an Azure Databricks data source (Unity Catalog, or Hive metastore) into a Data Map collection and (with -ConfigureScan) prepares a scoped, one-time scan using a named scan rule set. SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and only prints the exact registration/scan payload it WOULD send — it changes nothing. Pass -Commit to apply. No Databricks secret is ever placed in this script: the scan credential (a service principal or PAT) is stored in Azure Key Vault and referenced as a Purview credential BY NAME. HOW TO RUN Preview a Unity Catalog registration (default, no change): powershell -ExecutionPolicy Bypass -File .\DGDBX-Register-DatabricksSource.ps1 -PurviewAccount contoso-purview `-Collection DGDBX-Collection-Pilot -SourceName`

dbx-lakehouse-01 -WorkspaceUrl https://adb-1234567890123456.7.azuredatabricks.net -Metastore contoso-uc-metastore ` -Credential DGDBX-Cred-Databricks-KV Apply + configure a scoped scan of one catalog/schema: ... same args ... -ConfigureScan -Catalog contoso_prod -Schema finance -ScanRuleSet DGDBX-ScanRuleSet-Pilot -Commit For a legacy workspace use -Kind AzureDatabricksHiveMetastore (verify the exact connector name). Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a one-time scan definition is created scoped to the given catalog/schema; the script prints the resulting object names. Run Test connection in the portal, then trigger/monitor the scan; review with DGDBX-Report.ps1. .NOTES Purview — Databricks Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (Azure Databricks Unity Catalog vs. Hive metastore), the credentialType the connector accepts, the scan-scope shape, and the current scan API versions for your tenant in Microsoft's docs — the Data Governance connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$WorkspaceUrl, # e.g. https://adb-xxxx.NN.azuredatabricks.net [ValidateSet('AzureDatabricksUnityCatalog','AzureDatabricksHiveMetastore')] [string]$Kind = 'AzureDatabricksUnityCatalog', # verify the exact connector kind for your tenant [string]$Metastore, # Unity Catalog metastore (name/id) — verify [Parameter(Mandatory = $true)][string]$Credential, # NAME of the Purview credential (Key Vault-backed) — no secret here [switch]$ConfigureScan, [string]$Catalog, # bounded scan scope: one catalog ... [string]$Schema, # ... and one schema (pilot-first) [string]$ScanRuleSet = 'DGDBX-ScanRuleSet-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Preview a Unity Catalog registration (default, no change): powershell -ExecutionPolicy Bypass -File .\DGDBX-Register-DatabricksSource.ps1 -PurviewAccount contoso-purview -Collection DGDBX-Collection-Pilot -SourceName dbx-lakehouse-01 -WorkspaceUrl https://adb-1234567890123456.7.azuredatabricks.net -Metastore contoso-uc-metastore ` -Credential DGDBX-Cred-Databricks-KV Apply + configure a scoped scan of one catalog/schema: ... same args ... -ConfigureScan -Catalog contoso_prod -Schema finance -ScanRuleSet DGDBX-ScanRuleSet-Pilot -Commit For a legacy workspace use -Kind AzureDatabricksHiveMetastore (verify the exact connector name). Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a one-time scan definition is created scoped to the given catalog/schema; the script prints the resulting object names. Run Test connection in the portal, then trigger/monitor the scan; review with DGDBX-Report.ps1. .NOTES Purview — Databricks Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (Azure Databricks Unity Catalog vs. Hive metastore), the credentialType the connector accepts, the scan-scope shape, and the current scan API versions for your tenant in Microsoft's docs — the Data Governance connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$WorkspaceUrl, # e.g. https://adb-xxxx.NN.azuredatabricks.net [ValidateSet('AzureDatabricksUnityCatalog','AzureDatabricksHiveMetastore')] [string]$Kind = 'AzureDatabricksUnityCatalog', # verify the exact connector kind for your tenant [string]$Metastore, # Unity Catalog metastore (name/id) — verify [Parameter(Mandatory = $true)][string]$Credential, # NAME of the Purview credential (Key Vault-backed) — no secret here [switch]$ConfigureScan, [string]$Catalog, # bounded scan scope: one catalog ... [string]$Schema, # ... and one schema (pilot-first)

[string]$ScanRuleSet = 'DGDBX-ScanRuleSet-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

DGDBX-Report.ps1

Purpose: Reports the health of the governed Databricks estate for a scope and exports it to CSV: latest scan run status per Azure Databricks source, counts of classified vs unclassified lakehouse assets, ownership coverage, and lineage coverage (assets that have lineage relationships). Read-only — it queries the Purview data-plane REST / catalog search and changes nothing. Use it to review the pilot (Testing Guide) and to track coverage as the metastore is expanded. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGDBX-Report.ps1 -PurviewAccount contoso-purview ` -Collection DGDBX-Collection-Pilot -OutFile .\dgdbx-report.csv Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A console coverage summary AND a CSV (default .\dgdbx-report.csv) with one row per scanned asset — name, type, qualified name, classifications, owner, and whether lineage is present — plus a printed classification-, ownership-, and lineage-coverage percentage. PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — Databricks Data Governance. Read-only. Verify current search/scan/lineage API versions for your tenant; the Data Governance experience and its endpoints are evolving, and lineage depth is connector-dependent. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$OutFile = '.\dgdbx-report.csv', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGDBX-Report.ps1 -PurviewAccount contoso-purview ` -Collection DGDBX-Collection-Pilot -OutFile .\dgdbx-report.csv Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A console coverage summary AND a CSV (default .\dgdbx-report.csv) with one row per scanned asset — name, type, qualified name, classifications, owner, and whether lineage is present — plus a printed classification-, ownership-, and lineage-coverage percentage. PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — Databricks Data Governance. Read-only. Verify current search/scan/lineage API versions for your tenant; the Data Governance experience and its endpoints are evolving, and lineage depth is connector-dependent. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$OutFile = '.\dgdbx-report.csv', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGDBX-Set-Scope.ps1

Purpose: Captures a scope manifest for the Databricks pilot (or an expansion wave): the target collection, the workspace, the in-scope catalog/schema to scan first (with owner/steward and the credential name), the scan rule set intent, and the scan schedule. SAFE BY DEFAULT: it always writes a LOCAL JSON manifest (no tenant change) and only PREVIEWS the schedule it WOULD apply to the scan; it holds NO Databricks secret (the credential is referenced by name). Pass -Commit to actually set the scan schedule in Purview via the data-plane REST API — do this only after the scope has validated. HOW TO RUN Preview (default — writes local manifest, prints the schedule it would set, no tenant change): powershell -ExecutionPolicy Bypass -File .\DGDBX-Set-Scope.ps1 -Collection DGDBX-Collection-Pilot `-WorkspaceUrl https://adb-1234567890123456.7.azuredatabricks.net` -Catalog contoso_prod -Schema finance -Credential DGDBX-Cred-Databricks-KV ` -Recurrence Weekly -OutFile .\dgdbx-pilot-scope.json Apply the schedule to an existing scan (only after validation): ... same args ... -PurviewAccount contoso-purview -SourceName dbx-lakehouse-01 -ScanName dbx-lakehouse-01-scan-pilot -Commit EXPECTED RESULTS A dgdbx-pilot-scope.json manifest (collection, workspace, catalog/schema, owner/steward placeholders, credential name, rule set, schedule)

and a printed summary. With -Commit: the scan's recurring trigger is set (HTTP 200/201) and the script prints the result. .NOTES Purview — Databricks Data Governance. Local manifest is always safe; the schedule write is gated behind -Commit. Names are white-label placeholders. Verify the current scan-trigger/schedule API shape and api-version for your tenant before -Commit — the Data Governance experience is evolving. No secrets. > [CmdletBinding()] param([string]$Collection = 'DGDBX-Collection-Pilot', [string]$WorkspaceUrl, # e.g. https://adb-xxxx.NN.azuredatabricks.net [string]$Catalog, # bounded pilot scope: one catalog ... [string]$Schema, # ... and one schema [string]$Credential = 'DGDBX-Cred-Databricks-KV', # NAME of the Purview credential (Key Vault-backed) [string]$ScanRuleSet = 'DGDBX-ScanRuleSet-Pilot', [ValidateSet('OneTime','Daily','Weekly','Monthly')] [string]$Recurrence = 'OneTime', # keep OneTime until the scope validates [string]$OutFile = '.\dgdbx-pilot-scope.json', --- the following are only needed with -Commit (to set the schedule on an existing scan) --- [string]$PurviewAccount, [string]$SourceName, [string]$ScanName, [switch]$Commit)

How to run: Preview (default — writes local manifest, prints the schedule it would set, no tenant change): powershell -ExecutionPolicy Bypass -File .\DGDBX-Set-Scope.ps1 -Collection DGDBX-Collection-Pilot `-WorkspaceUrl https://adb-1234567890123456.7.azuredatabricks.net` -Catalog contoso_prod -Schema finance -Credential DGDBX-Cred-Databricks-KV ` -Recurrence Weekly -OutFile .\dgdbx-pilot-scope.json Apply the schedule to an existing scan (only after validation): ... same args ... -PurviewAccount contoso-purview -SourceName dbx-lakehouse-01 -ScanName dbx-lakehouse-01-scan-pilot -Commit EXPECTED RESULTS A dgdbx-pilot-scope.json manifest (collection, workspace, catalog/schema, owner/steward placeholders, credential name, rule set, schedule) and a printed summary. With -Commit: the scan's recurring trigger is set (HTTP 200/201) and the script prints the result. .NOTES Purview — Databricks Data Governance. Local manifest is always safe; the schedule write is gated behind -Commit. Names are white-label placeholders. Verify the current scan-trigger/schedule API shape and api-version for your tenant before -Commit — the Data Governance experience is evolving. No secrets. > [CmdletBinding()] param([string]$Collection = 'DGDBX-Collection-Pilot', [string]$WorkspaceUrl, # e.g. https://adb-xxxx.NN.azuredatabricks.net [string]$Catalog, # bounded pilot scope: one catalog ... [string]$Schema, # ... and one schema [string]$Credential = 'DGDBX-Cred-Databricks-KV', # NAME of the Purview credential (Key Vault-backed) [string]$ScanRuleSet = 'DGDBX-ScanRuleSet-Pilot', [ValidateSet('OneTime','Daily','Weekly','Monthly')] [string]$Recurrence = 'OneTime', # keep OneTime until the scope validates [string]$OutFile = '.\dgdbx-pilot-scope.json', --- the following are only needed with -Commit (to set the schedule on an existing scan) --- [string]$PurviewAccount, [string]$SourceName, [string]$ScanName, [switch]$Commit)

DGDBX-Test-Readiness.ps1

Purpose: Verifies prerequisites and the pilot gate in one view: the Purview account endpoint is reachable, the least-privilege roles (especially Data source admin) are present on the collection, an integration runtime is registered (managed or self-hosted for a private workspace), a Key Vault connection is present, the Azure Databricks source is registered and (where scanned) its last run Succeeded, classification coverage is > 0, and scanned assets have owners. Read-only — it makes no change. Use it to confirm readiness before registering, and again in the Validate step to decide whether the pilot passes before expanding. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGDBX-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGDBX-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP table across the readiness/gate signals and an overall verdict (PILOT PASS only when all gate signals PASS). Any GAP names the requirement to remediate before expanding scope. .NOTES Purview — Databricks Data Governance. Read-only gate check; no writes. Verify current Purview data-plane API

versions for your tenant — the Data Governance experience and the Azure Databricks connector evolve. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGDBX-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGDBX-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGDBX-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP table across the readiness/gate signals and an overall verdict (PILOT PASS only when all gate signals PASS). Any GAP names the requirement to remediate before expanding scope. .NOTES Purview — Databricks Data Governance. Read-only gate check; no writes. Verify current Purview data-plane API versions for your tenant — the Data Governance experience and the Azure Databricks connector evolve. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGDBX-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")